

操作系统

第二章 进程管理

Unix V6++ 的内存管理子系统（下） 内存分配

同济大学计算机系

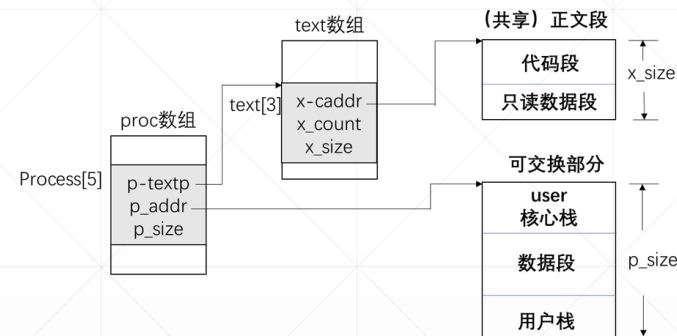
Part 1、Unix V6/Unix V6++ 内存分配



要解决的问题：登记空闲分区，为正文段和可交换部分分配内存。

使用的内存管理策略：

- 连续内存管理方式：共享正文段在物理内存和盘交换区上连续存放。可交换部分也是如此。
- 内存分配算法：
 - 系统维护一个空闲分区表，登记空闲分区。空闲分区按起始地址递增排序。
 - 内存分配操作：遍历空闲分区表，为正文段或可交换部分分配一块足够大的空闲区。如果空闲区尺寸大于所需，剩余部分是新的空闲分区，回收。
 - 内存回收操作：将回收的空闲分区插入空闲分区表。必要时，合并相邻空闲分区。



一、理论

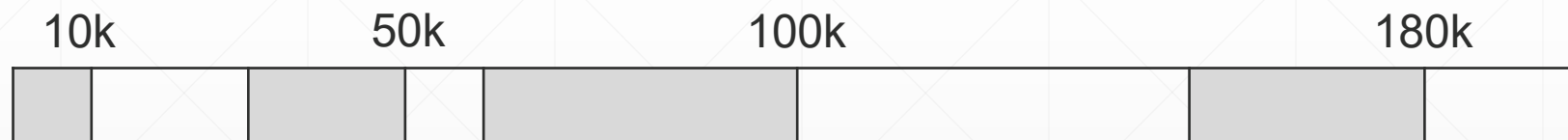
空闲分区 和 空闲分区表

- 空闲分区表 map 管理内存中的空闲区。
 - 空闲分区表管理空闲区。每个表项管理一个空闲区，登记首地址和长度。

空闲分区表 map

编号	首地址	长度
1	10k	20k
2	50k	10k
3	100k	50k
4	180k	20k

200K物理内存



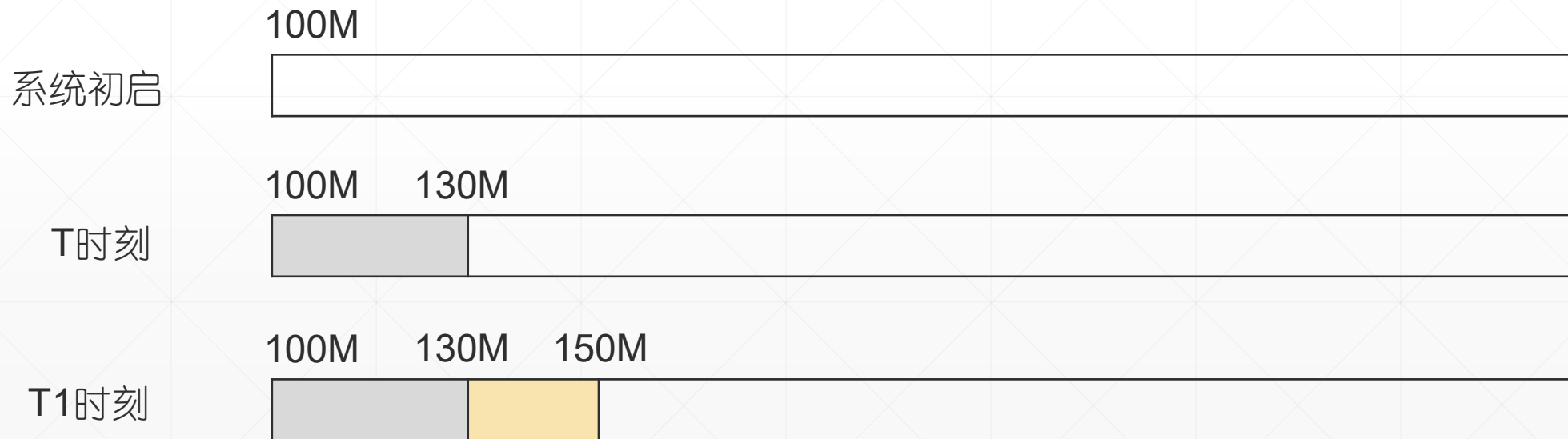
空闲区



已分配内存区中存有一个正文段或一个可交换部分，或者紧贴存放多个进程的图像。

例1：空闲分区的使用，假设用户区首地址为100M，长度是200M字节

- 启动时，没有进程。用户区空。map只有一个空闲区：首地址100M，长度200M。
- T时刻，创建进程A，长度为30M字节。
 - 进程A，首地址100M。
 - 1个空闲区，首地址130M，长度170M。
- T1时刻，创建进程B，长度为20M字节。
 - 进程B，首地址130M。
 - 1个空闲区，首地址150M，长度150M。



- T2时刻，进程A终止。

- 2个空闲区

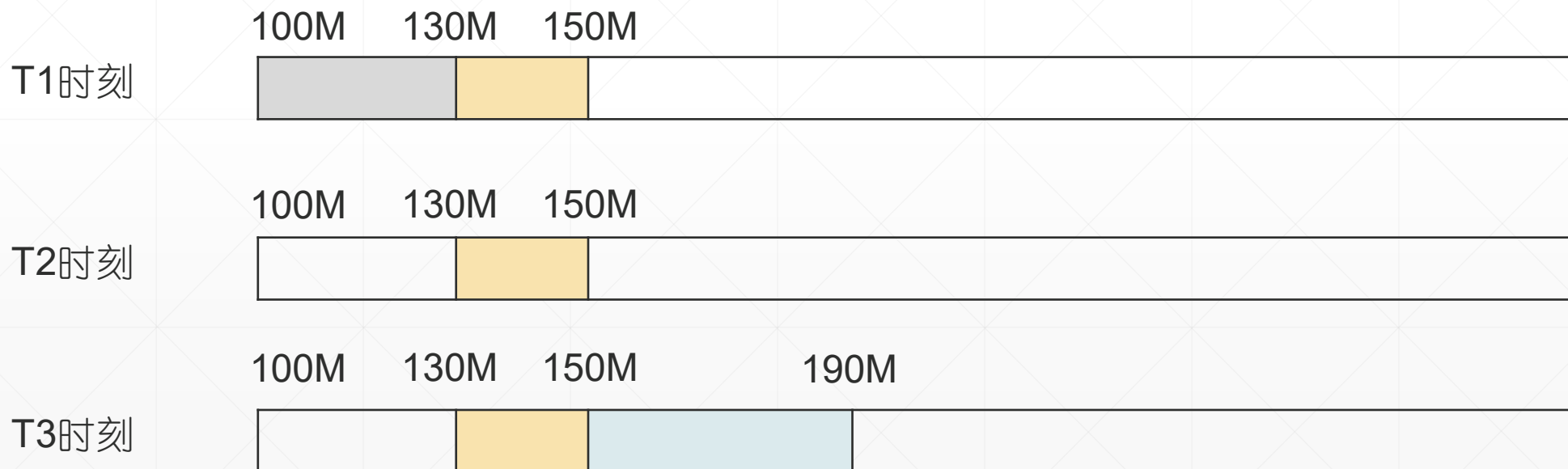
- 首地址100M，长度30M。
- 首地址150M，长度150M。

- T3时刻，创建进程C，长度为40M。

- 进程C，首地址150M。

- 2个空闲区

- 首地址100M，长度30M。
- 首地址190M，长度110M。



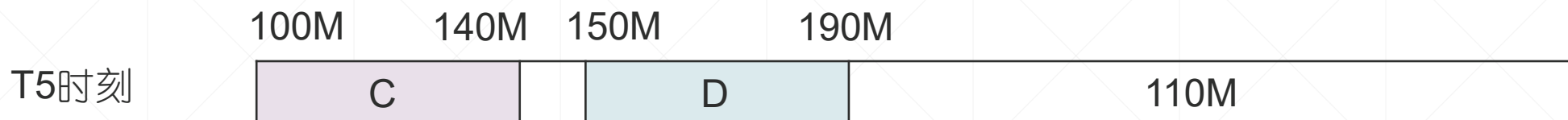
- T4时刻，进程B终止。
 - 2个空闲区
 - 首地址100M，长度50M。
 - 首地址190M，长度110M。



- T5时刻，创建进程D，长度为40M字节。
 - 进程D，首地址100M。
 - 2个空闲区
 - 首地址140M，长度10M。
 - 首地址190M，长度110M。



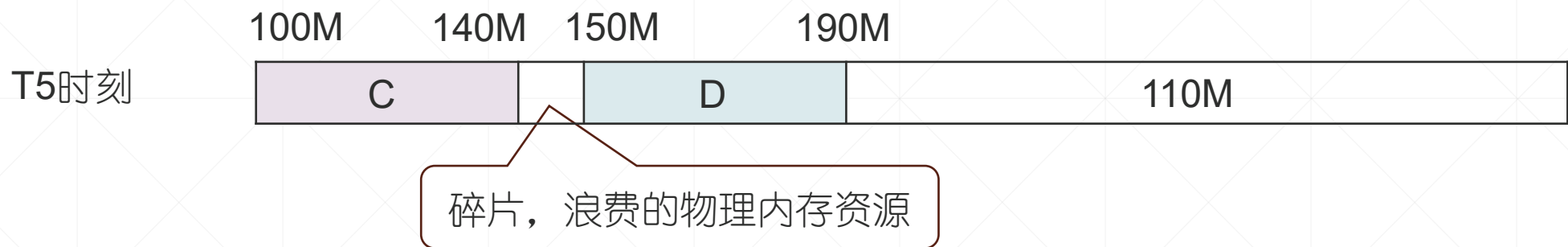
- T6时刻，创建进程E，长度为120M。
 - 没有能够容纳E的连续内存区域
 - 内存分配失败。需要把进程E的图像放在盘交换区上



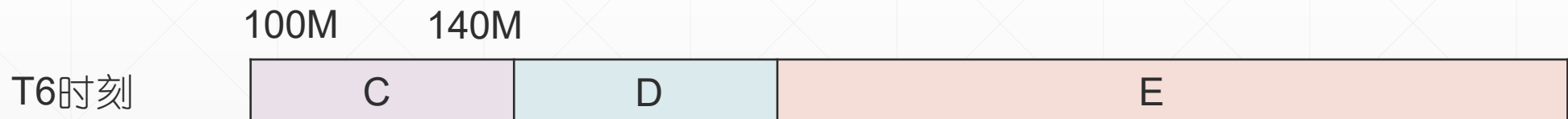
空闲内存，总量足够，但无法提供**120M**连续物理内存块。

10M小空间，装不进稍具体量的进程。所以，在进程**C** 或 **D**终止前，得不到利用，浪费物理内存。

碎片



碎片问题的解决：拼接。平移D，产生120M连续空间，装入E。



拼接非常耗时。

提高物理内存利用率（解决碎片问题），更有效的方法是：进程图像离散化，允许相邻逻辑页存放在不连续物理页框。



二、案例 Unix V6++的空闲资源管理

- 内存分配策略：连续内存管理方式、可变分区、首次适应
- 要求为进程图像分配连续物理内存，连续内存管理方式
- 可变分区：分区数量动态变化。
- 首次适应（first fit）：为进程分配的分区，是足够大的，首地址最小的分区。



1、Unix V6++ 的内存分配策略

全局对象 `Allocator Allocator::m_Instance`; 定义在 `src/mm/Allocator.cpp`

`unsigned long Allocator::Alloc(MapNode map[], unsigned long size)`

内存分配。 `map`, 空闲分区表; `size`, 分配的空间尺寸

`unsigned long Allocator::Free(MapNode map[], unsigned long size, unsigned long addrIdx)`

内存回收。 `map`, 空闲分区表; `size`, 回收的空间尺寸; `addrIdx`, 回收的空间首地址 (物理)



作业：阅读、注释源代码

```
unsigned long Allocator::Alloc(MapNode map[], unsigned long size)
{
    MapNode* pNode;
    unsigned long retIdx = 0;

    /* 若pNode->m_Size == 0则表示已经遍历到结尾 */
    for ( pNode = map; pNode->m_Size; pNode++)
    {
        if ( pNode->m_Size >= size )
        {
            retIdx = pNode->m_AddressIdx;
            pNode->m_AddressIdx += size;
            pNode->m_Size -= size;
            /* 当前内存正好分配完成，将该MapNode所在位置后面的MapNode都向前移动一个位置 */
            if ( pNode->m_Size == 0 )
            {
                MapNode* pNextNode = (pNode + 1);
                for ( ; pNextNode->m_Size; ++pNode, ++pNextNode)
                {
                    pNode->m_AddressIdx = pNextNode->m_AddressIdx;
                    pNode->m_Size = pNextNode->m_Size;
                }
                pNode->m_AddressIdx = pNode->m_Size = 0;
            }
            return retIdx;
        }
    }
    /* no match found */
    return 0;
}
```

```
unsigned long Allocator::Free(MapNode map[], unsigned long size, unsigned long addrIdx)
{
    MapNode* pNode;
    /* 首先，pNode指向正好是addrIdx下一块空闲区 */
    for ( pNode = map; pNode->m_AddressIdx <= addrIdx && pNode->m_Size != 0; ++pNode );
    /*
    * 1) pNode不是第一块，即不是在头部插入
    * 2) 需要free的数据块正好与pLastNode相邻
    * 3) 因为如果pNode是第一个话条件2)一定不会成立，
    * 因此可以保证pLastNode在条件内一定不会小于map的地址
    */
    MapNode* pLastNode = pNode - 1;
    if ( pNode > map && addrIdx == pLastNode->m_AddressIdx + pLastNode->m_Size )
    {
        pLastNode->m_Size += size;
        /* 这里处理与后面空闲块相邻的情况，需要合并后依次向前移动空闲块 */
        if ( addrIdx + size == pNode->m_AddressIdx )
        {
            pLastNode->m_Size += pNode->m_Size;
            for ( ++pLastNode, ++pNode; pNode->m_Size; ++pLastNode, ++pNode )
            {
                pLastNode->m_AddressIdx = pNode->m_AddressIdx;
                pLastNode->m_Size = pNode->m_Size;
            }
            pLastNode->m_AddressIdx = pLastNode->m_Size = 0;
        }
    }
}
```

```
/* 这里处理两种情况
* 1) 正好与pNode相邻且pNode不是无效的，只要修改pNode的地址Idx属性即可
* 2) 前后都不相邻，则需要将pNode及以后的节点依次向后移动
*/
else
{
    if ( addrIdx + size == pNode->m_AddressIdx && pNode->m_Size )
    {
        pNode->m_AddressIdx = addrIdx;
        pNode->m_Size += size;
    }
    else if ( size ) //合法性判断
    {
        /*处理条件2)的情况
        MapNode tmpNode1, tmpNode2;
        tmpNode1.m_AddressIdx = addrIdx;
        tmpNode1.m_Size = size;

        for ( ; pNode->m_Size; ++pNode )
        {
            tmpNode2.m_AddressIdx = pNode->m_AddressIdx;
            tmpNode2.m_Size = pNode->m_Size;

            pNode->m_AddressIdx = tmpNode1.m_AddressIdx;
            pNode->m_Size = tmpNode1.m_Size;

            tmpNode1.m_AddressIdx = tmpNode2.m_AddressIdx;
            tmpNode1.m_Size = tmpNode2.m_Size;
        }
        /* 将最后一个填入pNode中 */
        pNode->m_AddressIdx = tmpNode1.m_AddressIdx;
        pNode->m_Size = tmpNode1.m_Size;
    }
}
return 0;
}
```



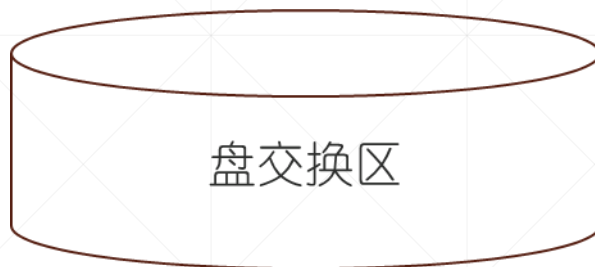
Unix V6++ 系统的内存资源 和 动态内存分配器



物理内存	存放的内容	动态内存分配器	内存分配粒度
[0, 1M)	-	-	-
[1M, 1.5M)	内核代码 & 数据（内核静态区域）	-	-
[1.5M, 2M)	内核堆（暂时未用）	g_KernelAllocator	字节
[2M, 4M-4K), 页表区	页表（兼为系统提供动态内存）	g_KernelPageManager	页
[4M-4K, 4M)	0#进程的PPDA	-	-
[4M, 32M), 用户区	进程的共享正文段 和 可交换部分	g_UserPageManager	页

src/kernel/Kernel.cpp中定义的全局对象

Unix V6++ 系统的盘交换区 和 动态分配器



磁盘扇区	存放的内容	动态内存分配器	分配粒度
[18000, 200000)	共享正文段复本（全部） 部分可交换部分（物理内存不够用时）	g_SwapperManager	扇区（512字节）

src/kernel/Kernel.cpp中定义的全局对象



PageManager, 以页为粒度分配物理内存

```
class PageManager                                     src/include/PageManager.h
{
public:
    static unsigned int PHY_MEM_SIZE;                // 物理内存大小, 系统启动时根据物理内存大小设置
    static const unsigned int PAGE_SIZE              = 0x1000; // 页大小
    static const unsigned int MEMORY_MAP_ARRAY_SIZE  = 0x200; // map数组长度
    static const unsigned int KERNEL_MEM_START_ADDR = 0x100000; // 内核映像从1M物理内存开始
    static const unsigned int KERNEL_SIZE            = 0x80000; // 内核映像长度不超过0.5M

public:
    PageManager(Allocator* allocator); // 构造函数
    virtual ~PageManager();
    int Initialize(); // 初始化
    unsigned long AllocMemory(unsigned long size); // 内存分配
    unsigned long FreeMemory(unsigned long size, unsigned long memoryStartAddress); // 内存回收

public:
    MapNode map[PageManager::MEMORY_MAP_ARRAY_SIZE]; // 空闲分区表, 512个元素

private:
    Allocator* m_pAllocator; // 内存分配策略 (连续内存管理方式, 可变分区分配, 首次适应算法)
};
```



初始化

src/mm/PageManager.cpp

```
PageManager::PageManager(Allocator* allocator)
{
    this->m_pAllocator = allocator;
}

int PageManager::Initialize()
{
    for ( unsigned int i = 0; i < MEMORY_MAP_ARRAY_SIZE; i++ )
    {
        this->map[i].m_AddressIdx = 0;
        this->map[i].m_Size = 0;
    }
    return 0;
}
```




分配 和 回收

src/mm/PageManager.cpp

// 4096字节上取整后, 调用Alloc分配内存

```
unsigned long PageManager::AllocMemory(unsigned long size)
{
    return this->m_pAllocator->Alloc(this->map,
                                     (size + (PAGE_SIZE - 1)) / PAGE_SIZE ) * PAGE_SIZE;
}

unsigned long PageManager::FreeMemory(unsigned long size, unsigned long startAddress)
{
    return this->m_pAllocator->Free(this->map,
                                    (size + (PAGE_SIZE - 1)) / PAGE_SIZE, startAddress / PAGE_SIZE);
}
```



用户区(g_UserPageManager)

src/include/PageManager.h

```
class UserPageManager : public PageManager
{
public:
    /* static const member */
    static const unsigned int USER_PAGE_POOL_START_ADDR = 0x400000;    /* 用户区起始地址 */
    /* static member */
    static unsigned int USER_PAGE_POOL_SIZE;    /* 用户区尺寸：由内核初始化时进行设置 */

public:
    UserPageManager(Allocator* allocator);
    int Initialize();    /* 初始化MapNode map[0]为用户物理页区起始地址、大小 */
};
```



```
UserPageManager::UserPageManager(Allocator* allocator)
    :PageManager(allocator)
```

src/mm/PageManager.cpp

```
{
}
```

```
int UserPageManager::Initialize()
```

```
{
```

```
    PageManager::Initialize();
```

```
    this->map[0].m_AddressIdx =
```

```
        USER_PAGE_POOL_START_ADDR / PageManager::PAGE_SIZE;
```

```
    this->map[0].m_Size =
```

```
        USER_PAGE_POOL_SIZE / PageManager::PAGE_SIZE;
```

```
    return 0;
```

```
}
```



页表区(g_KernelPageManager)

src/include/PageManager.h

```
class KernelPageManager : public PageManager
{
public:
```

```
    /* 0x200#物理页框，系统页目录
    * 0x201#物理页框，系统核心页表
    * 0x202#，0x203#物理页框，系统用户页表
    * 相对表，从0x204#页框开始分配
    */
```

```
    static const unsigned int KERNEL_PAGE_POOL_START_ADDR = 0x204000;
    static const unsigned int KERNEL_PAGE_POOL_SIZE = 0x200000 - 0x4000;
```

```
public:
```

```
    KernelPageManager(Allocator* allocator);
    int Initialize();
```

.....

其余的动态内存分配器，自行探索

- 页表区，g_KernelPageManager。

KernelPageManager，PageManager的子类，以页为单位分配内存。

物理内存	存放的内容	动态内存分配器	内存分配粒度
[0, 1M)	-	-	-
[1M, 1.5M)	内核代码 & 数据（内核静态区域）	-	-
[1.5M, 2M)	内核堆（暂时未用）	g_KernelAllocator	字节
[2M, 4M-4K)，页表区	页表（兼为系统提供动态内存）	g_KernelPageManager	页
[4M-4K, 4M)	0#进程的PPDA	-	-
[4M, 32M)，用户区	进程的共享正文段和 可交换部分	g_UserPageManager	页

磁盘扇区	存放的内容	动态内存分配器	分配粒度
[18000, 200000)	共享正文段复本（全部） 部分可交换部分（物理内存不够用时）	g_SwapperManager	扇区（512字节）

评论

物理内存	存放的内容	动态内存分配器	内存分配粒度
[0, 1M)	-	-	-
[1M, 1.5M)	内核代码 & 数据（内核静态区域）	-	-
[1.5M, 2M)	内核堆（暂时未用）	g_KernelAllocator	字节
[2M, 4M-4K), 页表区	页表（兼为系统提供动态内存）	g_KernelPageManager	页
[4M-4K, 4M)	0#进程的PPDA	-	-
[4M, 32M), 用户区	进程的共享正文段和 可交换部分	g_UserPageManager	页

磁盘扇区	存放的内容	动态内存分配器	分配粒度
[18000, 200000)	共享正文段复本（全部） 部分可交换部分（物理内存不够用时）	g_SwapperManager	扇区（512字节）

连续存储管理方式的分配算法

性能指标 (1) 内存利用率 (2) 能否保留大分区 (3) 分配、回收操作的速度



- 首次适应 first fit。Unix V6使用的方法。
 - 碎片集中在低地址端
 - 高地址端保留大分区，对大尺寸进程有利

缺点：碎片多，影响内存利用率。
分配回收操作，扫描数组元素多。
优点：保留大分区。

连续存储管理方式的分配算法

- 下次适应 next fit。
 - 空闲分区表是个循环队列，current是循环队列的当前元素。每次分配
 - 从current的下个元素开始扫描，首次适应。
 - current停留在目标分区
 - 性能分析
 - 碎片少
 - 很难保留大分区

- 最佳适应 best fit.
 - 遍历空闲分区表，切割最小分区
 - 性能分析
 - 碎片最多
 - 最大程度保留大分区
- 最差适应 worst fit.
 - 遍历空闲分区表，切割最大分区
 - 性能分析
 - 碎片最小
 - 完全无法保留大分区

综合性能：first fit最佳

fork(), 为子进程分配相对表

SaveU(u.u_rsav); // for 进程切换, 缓

1、PageTable* pgTable = u.u_MemoryDescriptor.m_UserPageTableArray; // 这是父进程的页表, 记住首地址
u.u_MemoryDescriptor.Initialize(); // 为子进程分配相对表

2、if (NULL != pgTable) // 复制相对表

Utility::MemCopy((unsigned long)pgTable, (unsigned long)u.u_MemoryDescriptor.m_UserPageTableArray,
sizeof(PageTable) * MemoryDescriptor::USER_SPACE_PAGE_TABLE_CNT);

void MemoryDescriptor::Initialize()

管理页表区
g_KernelPageManager

```
{  
    ★ KernelPageManager& kernelPageManager = Kernel::Instance().GetKernelPageManager();  
    this->m_UserPageTableArray = (PageTable*)(kernelPageManager.AllocMemory(sizeof(PageTable) *  
        USER_SPACE_PAGE_TABLE_CNT)  
        + Machine::KERNEL_SPACE_START_ADDRESS);  
}
```



fork(), 为子进程分配可交换部分

```
u.u_procp = child;
```

```
UserPageManager& userPageManager = Kernel::Instance().GetUserPageManager(); // g_UserPageManager
```

```
unsigned long srcAddress = current->p_addr; // 父进程可交换部分首地址, 物理地址
```

```
unsigned long desAddress = userPageManager.AllocMemory(current->p_size); // 为子进程可交换部分分配物理内存
```

```
if ( desAddress == 0 ) // 失败
```

```
{
```

父进程可交换部分, 复制一份在盘交换区。这是子进程的可交换部分, 子进程的p_addr是盘交换区的起始扇区号。
这个子进程是盘交换区上的进程, p_flag SLOAD位清0。

```
}
```

```
else // 成功
```

```
{
```

```
    int n = current->p_size;
```

```
    child->p_addr = desAddress; // 设置子进程的p_addr
```

```
    while (n--)
```

```
    {
```

```
        Utility::CopySeg(srcAddress++, desAddress++); // 复制父进程可交换部分, 每次一个字节
```

```
    } // CopySeg(), 物理空间复制数据
```

```
}
```

情景分析，例题1



- T0时刻，用户区如图所示。
- PA进程代码段3页，数据段1页，堆栈段1页； $x_caddr=0x402000$ ， $p_addr=0x407000$
- PA创建子进程PB。

(1) PB进程， $x_caddr = (\quad)$ ， $p_addr = (\quad)$

(2) 画 PA的相对表, PB的相对表

正文段

代码, 第0页
代码, 第1页

可交换部分

PPDA
数据
用户栈

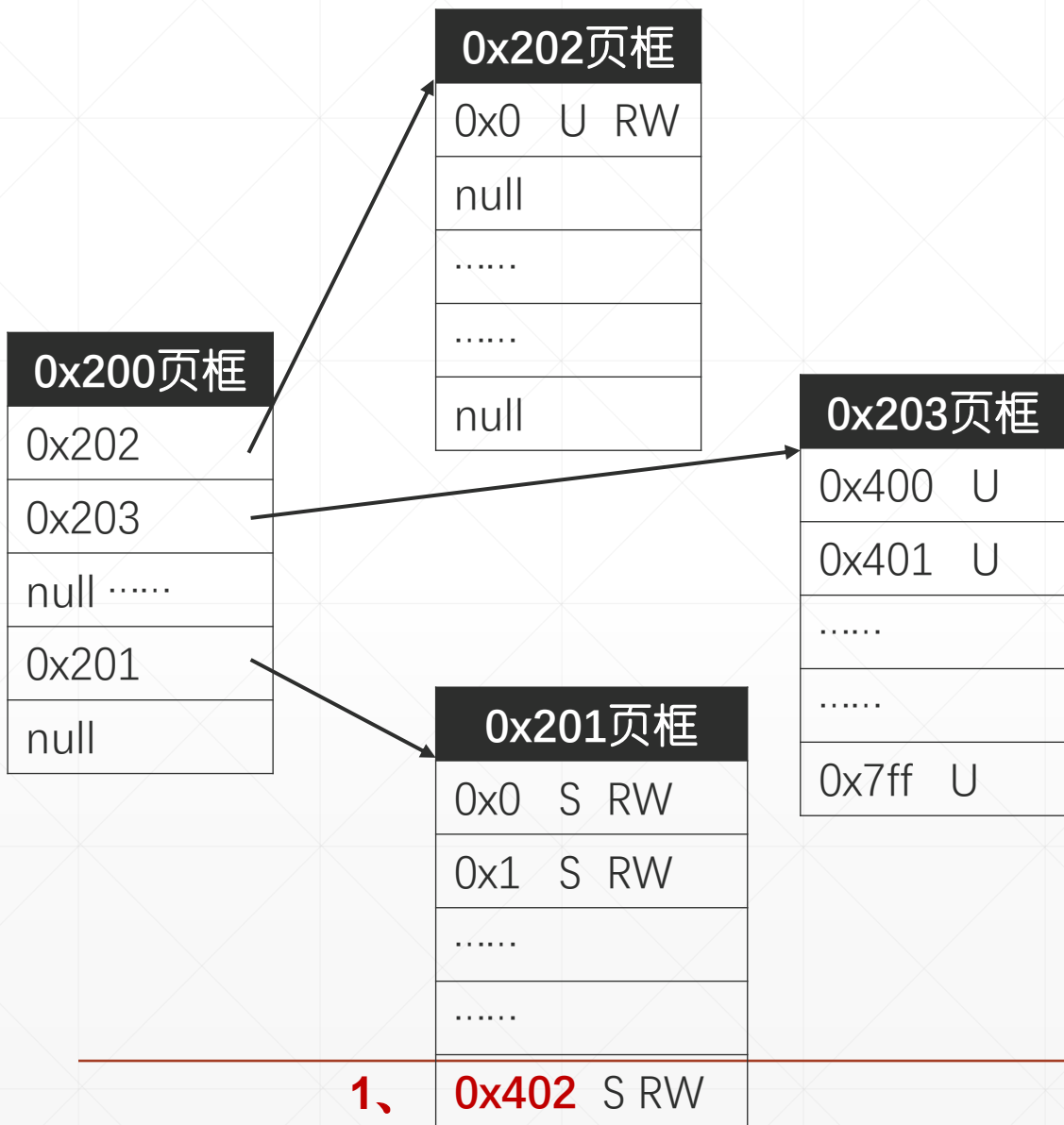
PA的相对表

0	null
1	null
.....	
1024	null
1025	0 U RO P
1026	1 U RO P
1027	1 U RW P
1028	null
.....	
.....	null
2047	2 U RW P

PB的相对表

null
null
.....
null
0 U RO P
1 U RO P
1 U RW P
null
.....
null
2 U RW P

(3) 调度PA，执行前为其建立地址映射关系



PA进程代码段3页，数据段1页，堆栈段1页；
x_caddr=0x402000, p_addr=0x407000

1、SwchUStruct(selected);

p_addr > 12, 填入核心页表1023#PTE的 base 字段，建立PPDA区的地址映射关系。

```
#define SwchUStruct(p) \
Machine::Instance().GetKernelPageTable().m_Entrys[Kernel::USER_PAGE_INDEX].m_PageBaseAddress \
= (p)->p_addr / PageManager::PAGE_SIZE; \
FlushPageDirectory();
```

1023 PTE中的base

2、User& newu = Kernel::Instance().GetUser();

newu 是装在物理页框0x402里，PA进程的user结构

3、newu.u_MemoryDescriptor.MapToPageTable();

以PA的相对表为模板，写0x202,0x203#物理页框